

VULNPULSE

Cyber Vulnerability Intelligence Lakehouse

Phase 1 Project Proposal | Data Analysis and Visualization

Project proposition: Build a storage-aware Apache Spark data pipeline that combines the National Vulnerability Database (NVD) with CISA Known Exploited Vulnerabilities (KEV) data, transforms them through Bronze/Silver/Gold layers, and delivers a security intelligence dashboard.

Item	Planned value
Team	2 students
Primary platform	Databricks Free Edition (current replacement for legacy Community Edition)
BI layer	Power BI
Primary source	NVD 2.0 JSON feeds + NVD CVE API 2.0
Enrichment	CISA Known Exploited Vulnerabilities (KEV)
Historical baseline	2020-2026 NVD yearly feeds
Incremental pattern	Watermark-driven NVD API query on `lastModified`
GitHub	https://github.com/kashishfatima999/VulnPulse
Phase 2 target date	10 October 2026
Phase 3 target date	24 October 2026

Prepared for Phase 1 submission | Design baseline date: 25 September 2026

1. Executive Summary

VulnPulse is an end-to-end vulnerability intelligence data platform designed as a practical Data Engineering workload rather than a static analytics exercise. It will ingest a historical baseline from NVD, process subsequent vulnerability changes incrementally, enrich the records with CISA KEV exploitation-status metadata, and produce compact analytical datasets for Power BI.

The project is deliberately scoped so Apache Spark is justified by nested JSON, large historical records, multi-source joins, schema normalization and incremental processing, while persistent storage remains controlled. Full NVD feeds are not stored in GitHub and are not repeatedly reprocessed after the initial baseline. The repository stores only small source excerpts required for reproducibility and Phase 1 evidence.

Stand-out engineering feature: A pipeline audit/data-quality layer will record batch size, accepted/rejected records, checksums, watermarks and processing outcomes. This demonstrates observability and operational thinking in addition to dashboard building.

2. Domain and Project Concept

Domain: Cybersecurity vulnerability intelligence.

2.1 Problem Statement

Organizations receive vulnerability information from continuously changing public sources. A useful analytical system must distinguish a historical baseline from newly created or modified records, standardize inconsistent nested structures, identify affected vendors/products and prioritize vulnerabilities that are known to be exploited in the wild. VulnPulse turns these raw feeds into structured security intelligence.

2.2 Project Objectives

- **Build a reproducible full-load pipeline.** Load a controlled 2020-2026 NVD historical baseline.
- **Build a true incremental pipeline.** Use NVD `lastModified` date filtering with a persisted watermark so only changed records are fetched after the baseline.
- **Integrate a second public source.** Join CISA KEV data to identify vulnerabilities known to have been exploited in the wild.
- **Apply Medallion Architecture.** Preserve source fidelity in Bronze, normalize in Silver, and create analyst-ready Gold datasets.
- **Produce useful BI.** Answer vendor, severity, weakness and exploitation-priority questions in Power BI.

3. Data Sources and Exact References

Source	Purpose	Exact reference	Load role
NVD	Core vulnerability records, CVSS, CWE, affected software, dates, references	https://nvd.nist.gov/	Primary
NVD 2.0 feeds	Historical yearly JSON 2.0 feeds	https://nvd.nist.gov/feeds/json/cve/2.0/	Full load

Source	Purpose	Exact reference	Load role
NVD CVE API 2.0	Date-filtered CVE updates using `lastModStartDate`/`lastModEndDate`	https://services.nvd.nist.gov/rest/json/cves/2.0	Incremental
NVD developer documentation	API parameters and response model	https://nvd.nist.gov/developers/vulnerabilities	Technical reference
CISA KEV	Known-exploited enrichment, date added, due date, ransomware association, action	https://www.cisa.gov/known-exploited-vulnerabilities-catalog	Enrichment
CISA KEV JSON	Machine-readable KEV feed	https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json	Enrichment payload
CISA KEV GitHub mirror	Audit-friendly mirror and change history	https://github.com/cisagov/kev-data	Optional audit/fallback
VulnPulse repository	Version control and required small sample files	https://github.com/kashishfatima999/VulnPulse	Project repo

Source rationale: NVD supplies the broad vulnerability universe; CISA KEV adds a high-value exploitation signal. NVD is kept as the source of truth for the core CVE record, while CISA is treated as enrichment.

4. Ingestion Pattern: Full Load + Incremental Load

4.1 Full Load

The historical baseline will cover NVD yearly JSON 2.0 feeds from 2020 through 2026 at project design time. The official file naming pattern is:

`https://nvd.nist.gov/feeds/json/cve/2.0/nvdcve-2.0-{YEAR}.json.gz`

A July 2026 public mirror of these NVD 2.0 feeds reported the following compressed sizes for 2020-2026: approximately 13.3, 16.0, 17.4, 19.2, 23.7, 22.5 and 20.2 MiB respectively, for about 131.5 MiB total compressed. Feed sizes change as records are added or modified, so the final proposal evidence will record the actual size when the team captures the baseline.

4.2 Incremental Load

After the historical baseline, the pipeline will request only CVEs modified since the previous successful watermark. The NVD CVE API supports `lastModStartDate` and `lastModEndDate` date filters and paginated results. The project will store the last successful `lastModified` timestamp, use a small overlap window, and deduplicate by CVE ID plus last-modified timestamp.

Example: `GET https://services.nvd.nist.gov/rest/json/cves/2.0?lastModStartDate=<START>&lastModEndDate=<END>&resultsPerPage=2000&startIndex=0`

This is preferred over repeatedly downloading the historical baseline. The official NVD modified feed is retained as an operational fallback/reference: `https://nvd.nist.gov/feeds/json/cve/2.0/nvdcve-2.0-modified.json.gz`.

4.3 Planned End-to-End Flow

Stage	What happens	Storage-conscious rule
Source acquisition	Capture NVD yearly feeds for baseline; capture NVD date-filtered incremental payloads; capture CISA KEV snapshot.	Do not keep duplicate copies after successful landing.
Bronze	Preserve source payloads with ingestion metadata, source, batch ID and checksum.	Keep compressed source landing only as long as operationally needed.
Silver	Flatten nested JSON, standardize dates/types, normalize vendors/products/CWEs, validate quality, handle identifying source metadata.	Keep only analytical columns in curated tables.
Gold	Aggregate trends, vendor exposure, CWE trends, KEV-response timing, watchlist and pipeline audit.	Gold tables are compact and dashboard-oriented.
BI	Power BI consumes only Gold outputs.	Do not connect Power BI to raw data.

5. Data Samples, Volume and Frequency

Phase 1 requires actual raw samples for both load types. The repository includes a sample-collection script that downloads small excerpts from the real sources and writes them to `data/samples/`. This avoids fabricating sample payloads and avoids putting full feeds into GitHub.

Artifact	Planned contents	Target GitHub size
nvd_full_load_sample.json	10 records extracted from the official 2026 NVD yearly JSON 2.0 feed.	< 1 MB
nvd_incremental_load_sample.json	10 records from a real 24-hour NVD API `lastModStartDate`/`lastModEndDate` query.	< 1 MB
cisa_kev_sample.json	10 current CISA KEV records.	< 1 MB
source_manifest.json	Capture timestamps, source URLs, record counts, sample sizes and hashes where applicable.	Tiny

5.1 Volume Estimate

Component	Phase 1 planning estimate	Control
NVD full baseline 2020-2026	~132 MiB compressed (based on July 2026 feed-size observations); re-measure at capture time.	Do not commit to GitHub.
NVD daily incremental	Plan for roughly sub-MB to low-single-digit MB compressed batches; measure the first several runs and record actual values.	Watermark + pagination + deduplication.
CISA KEV snapshot	~1.7 MB JSON in the current CISA-maintained GitHub mirror; small enough to refresh frequently.	Replace/refresh rather than accumulating copies.

Component	Phase 1 planning estimate	Control
GitHub samples	Intentionally tiny.	Hard repository rule: no full feeds, Parquet dumps, or Delta exports.

The purpose of the estimate is not to maximize raw volume. It is to demonstrate scalable processing while keeping the Free Edition workload small enough to be repeatable and safe.

6. Security, PII and Sensitive Data

PII means Personally Identifiable Information: data that can identify or meaningfully identify a person, such as an email address, phone number or personal identifier. VulnPulse does not intentionally collect customer-level or personal-user records.

A review is still necessary because NVD records include `sourceIdentifier`, and examples of NVD records show this field can contain email-like addresses used for source attribution. These addresses are not needed for the project analytics. The project will therefore treat them as potentially identifying source metadata and remove them from analytical Silver/Gold tables.

Field / pattern	Risk treatment	Reason
NVD sourceIdentifier	Detect email-like value; drop from Silver/Gold; optionally keep boolean `source_identifier_present`.	Not required for security analytics and may contain email-like contact metadata.
Vendor / product	Keep.	Public software metadata, core to the analysis.
CVE ID / CWE / CVSS	Keep.	Public vulnerability intelligence, core measures.
Reference URLs	Keep for analysis/audit where required; do not crawl them automatically in Phase 1.	Useful provenance, but not required for the first pipeline.
API keys / credentials	Never store.	Secrets must remain in local environment variables or Databricks secrets.

7. High-Level Medallion Data Model

7.1 Bronze Layer - Raw and Reproducible

Bronze preserves the source payload with minimal interpretation and attaches operational metadata: batch ID, source name, source file/request, load type, ingestion timestamp and source checksum. This supports traceability and reprocessing without forcing the analytical layers to depend on raw files.

7.2 Silver Layer - Cleansed and Normalized

Table	Grain	Main fields
silver_cve	One current normalized record per CVE	cve_id, published_at, last_modified_at, status, description, CVSS score/severity, attack metrics, primary CWE
silver_affected_product	One CVE-product relationship	cve_id, vendor, product, version range, vulnerable flag
silver_cwe	One CVE-CWE relationship	cve_id, cwe_id
silver_reference	One CVE-reference relationship	cve_id, reference_url, reference_type

Table	Grain	Main fields
silver_kev	One KEV entry per CVE	cve_id, date_added, due_date, ransomware_use, required_action

Core Silver transformations: flatten nested JSON, keep English description, cast dates/timestamps, standardize severity, derive numeric CVSS fields, split vendor/product relationships, normalize CWE identifiers, remove duplicate CVE records, handle missing values and validate numeric/date domains.

7.3 Gold Layer - Analyst Ready

Gold table	Purpose	Representative metrics
gold_vulnerability_trends	Time-based vulnerability landscape	CVEs by month/year, severity mix, critical/high counts
gold_vendor_risk	Vendor/product exposure analysis	total CVEs, critical CVEs, KEV CVEs, average CVSS
gold_kev_response	Exploit-related response timing	days from NVD publication to CISA KEV catalog inclusion
gold_cwe_trends	Weakness trend analysis	CVE count, KEV count and severity by CWE over time
gold_watchlist	Compact latest-priority view	KEV flag, severity, due date, recency and project-defined priority rules
gold_pipeline_audit	Operational observability	records in/out, rejected rows, bytes, run time and watermarks

Important metric definition: `days\_to\_kev\_inclusion = CISA dateAdded - NVD published date`. This is a time-to-catalog-inclusion measure, not a claim that the first exploitation occurred on the catalog date.

8. Business Intelligence and Dashboard Plan

The final Power BI dashboard will be designed for a security analyst or vulnerability-management team. The goal is not to reproduce the NVD website; it is to combine sources and expose trends and prioritization signals produced by the Spark pipeline.

8.1 Business / Analytical Questions

- How is the volume and severity of reported vulnerabilities changing over time?
- Which vendors and products have the highest concentration of known exploited vulnerabilities?
- Which weakness categories (CWE) appear most frequently among exploited vulnerabilities?
- How long, on average, is the gap between NVD publication and KEV catalog inclusion?
- Which current vulnerabilities meet the project-defined watchlist rules?

8.2 Planned Visuals

Visual	What it shows	Gold source
Severity trend line	Monthly/quarterly CVE volume split by severity.	gold_vulnerability_trends
Vendor exposure bar chart	Top vendors by total/critical/KEV vulnerabilities.	gold_vendor_risk
Publication-to-KEV distribution	Distribution of days to KEV inclusion, with median and percentile indicators.	gold_kev_response

Visual	What it shows	Gold source
CWE heatmap	Weakness categories by time and KEV count.	gold_cwe_trends
Current watchlist table	CVE, vendor/product, severity, KEV status, due date and action.	gold_watchlist

9. Engineering Setup, Version Control and FinOps

9.1 Platform

The planned Spark environment is Databricks Free Edition. Databricks documents Free Edition as a no-cost, serverless, quota-limited environment for students and learning. The current Free Edition replaced the legacy Databricks Community Edition in 2025. Free Edition also restricts outbound internet access, so source acquisition may need to occur locally and then be uploaded to a Unity Catalog volume before Spark processing.

This project therefore separates source acquisition from Spark processing: a small collector script handles reproducible source capture; Databricks automates Bronze -> Silver -> Gold transformations after a controlled landing batch is available.

9.2 FinOps / Storage Guardrails

- **No full feeds in GitHub.** Only small source excerpts are versioned.
- **No duplicate persistent copies.** Staged raw feeds are deleted after successful landing when no longer needed.
- **No unnecessary full reruns.** After the baseline, the pipeline processes only records changed since the watermark.
- **No large local collections.** Do not `collect()` large DataFrames; aggregate in Spark and export only Gold-level outputs.
- **Compact analytical tables.** Silver/Gold retain analytical columns, not duplicate raw payloads and reference blobs.
- **Development ladder.** Test with 10 records, then 1 year, then 2020-2026; do not jump directly to the largest run.
- **Internal storage guardrail.** Target a persistent project footprint below ~2 GB. This is a conservative project target, not a claim about Databricks quota.

FinOps in plain English: Use only the compute and storage needed for the current batch. The biggest saving is avoiding repeated full-history processing after the first load.

10. GitHub Repository and Phase 1 Contents

Current repository:

<https://github.com/kashishfatima999/VulnPulse>

At design time, the public repository is a starter repository. Phase 1 should turn it into an evidence-first project repository with the following structure:

```
VulnPulse/
├── README.md
```

```

├── .gitignore
├── config/
│   └── config.example.yaml
├── docs/
│   ├── Phase1_Proposal.pdf
│   ├── Phase1_Proposal.docx
│   ├── DATA_SOURCES.md
│   ├── ARCHITECTURE.md
│   ├── PII_FINOPS.md
│   └── PHASE1_CHECKLIST.md
├── data/
│   └── samples/
│       ├── nvd_full_load_sample.json
│       ├── nvd_incremental_load_sample.json
│       ├── cisa_kev_sample.json
│       ├── source_manifest.json
│       └── README.md
└── scripts/
    └── collect_phase1_samples.py

```

Do not add: full NVD feeds, full CISA snapshots, Parquet/Delta exports, notebook output dumps, API keys, `.env` files or other large generated artifacts.

11. Phase 1 Rubric Coverage

Teacher requirement	VulnPulse evidence in this proposal/repository	Status at Phase 1
1. Domain & Source Identification	Cybersecurity vulnerability intelligence; NVD and CISA exact URLs; explicit full vs incremental pattern.	Covered by design; samples required before upload.
2. Data Samples & Volume	Sample collector script + file names + measured/estimated source size + frequency plan.	Proposal covered; generate actual samples before submission.
3. Security & Compliance	PII explanation, `sourceIdentifier` handling, no secrets in repo, source provenance.	Covered.
4. Medallion Modeling	Bronze/Silver/Gold tables and transformations.	Covered.
5. BI & Dashboard	Business questions, Gold outputs and planned visuals.	Covered.
6. Engineering & FinOps	GitHub structure, Databricks Free Edition, storage/compute guardrails, no giant repository artifacts.	Covered.

12. Semester Execution Plan

Period	Main deliverable	Storage/compute rule
Phase 1 - now	Proposal, sample evidence, source documentation, repository structure.	No large cloud processing yet.
Phase 2 - by 10 Oct 2026	Full ingestion + Bronze/Silver + incremental mechanism + data-quality checks, subject to later requirements.	Build progressively: samples -> one year -> full baseline.

Period	Main deliverable	Storage/compute rule
Phase 3 - by 24 Oct 2026	Gold layer + Power BI dashboard + end-to-end testing/documentation, subject to later requirements.	Refresh incrementally; export only compact dashboard-ready data.

13. Features That Make VulnPulse Stand Out

- **Multi-source enrichment.** NVD alone becomes much more useful when joined with CISA KEV.
- **True incremental design.** A persisted watermark avoids reprocessing the historical baseline.
- **Data-quality gate.** Schema, null, duplicate, date and CVSS validation occur before Gold.
- **Operational observability.** The pipeline audit table measures batch behavior and demonstrates Data Engineering maturity.
- **Clearly defined derived metric.** Time to KEV inclusion is transparent about what the data does and does not prove.
- **Storage-aware engineering.** The design explicitly protects GitHub and free-tier storage from raw-data accumulation.

14. Official / Supporting References

- [NVD - National Vulnerability Database](#)
- [NVD 2.0 JSON CVE feeds](#)
- [NVD CVE API 2.0](#)
- [NVD developer documentation](#)
- [NVD API key request](#)
- [CISA Known Exploited Vulnerabilities catalog](#)
- [CISA KEV JSON](#)
- [CISA KEV CSV](#)
- [CISA KEV GitHub mirror](#)
- [CISA KEV JSON schema](#)
- [Databricks Free Edition](#)
- [Databricks Free Edition limitations](#)
- [VulnPulse GitHub repository](#)

Phase 1 submission gate: before the proposal is submitted, run `python scripts/collect\_phase1\_samples.py`, inspect the generated samples, commit the four small sample/manifest files and all Phase 1 documentation, then verify that the public GitHub link opens without authentication.